



Figure 1: USB specifications for the pen drive

driver getting auto-loaded.

USB data transfer

"Time for USB data transfers. Let's build upon the USB device driver coded in our previous sessions, using the same handy JetFlash pen drive from Transcend, with vendor ID 0x058f and product ID 0x6387," said Pugs, enthusiastically.

USB, being a hardware protocol, forms the usual horizontal layer in the kernel space. And hence, for it to provide an interface to user-space, it has to connect through one of the vertical layers. As the character (driver) vertical has already been discussed, it is the current preferred choice for the connection with the USB horizontal, in order to understand the complete data transfer flow. Also, we do not need to get a free unreserved character major number, but can use the character major number 180, reserved for USB-based character device files. Moreover, to achieve this complete character driver logic with the USB horizontal in one go, the following are the APIs declared in `<linux/usb.h>`:

```
int usb_register_dev(struct usb_interface *intf, struct usb_
class_driver *class_driver);
void usb_deregister_dev(struct usb_interface *intf, struct
usb_class_driver *class_driver);
```

Usually, we would expect these functions to be invoked in the constructor and the destructor of a module, respectively. However, to achieve the hot-plug-n-play behaviour for the (character) device files corresponding to USB devices, these are instead invoked in the *probe* and *disconnect* callbacks, respectively. The first parameter in the above functions is the interface pointer received as the first parameter in both *probe* and *disconnect*. The second parameter, *struct usb_class_driver*, needs to be populated with the suggested device file name and the set of device file operations, before invoking *usb_register_dev*. For the

actual usage, refer to the functions *pen_probe* and *pen_disconnect* in the code listing of *pen_driver.c* below.

Moreover, as the file operations (write, read, etc) are now provided, that is exactly where we need to do the data transfers to and from the USB device. So, *pen_write* and *pen_read* below show the possible calls to *usb_bulk_msg()* (prototyped in `<linux/usb.h>`) to do the transfers over the pen drive's bulk end-points 0x01 and 0x82, respectively. Refer to the 'E' lines of the middle section in Figure 1 for the endpoint number listings of our pen drive. Refer to the header file `<linux/usb.h>` under kernel sources, for the complete list of USB core API prototypes for other endpoint-specific data transfer functions like *usb_control_msg()*, *usb_interrupt_msg()*, etc. *usb_rcvbulkpipe()*, *usb_sndbulkpipe()*, and many such other macros, also defined in `<linux/usb.h>`, compute the actual endpoint bit-mask to be passed to the various USB core APIs.

Note that a pen drive belongs to a USB mass storage class, which expects a set of SCSI-like commands to be transacted over the bulk endpoints. So, a raw read/write as shown in the code listing below may not really do a data transfer as expected, unless the data is appropriately formatted. But still, this summarises the overall code flow of a USB driver. To get a feel of a real working USB data transfer in a simple and elegant way, one would need some kind of custom USB device, something like the one available at <http://lddk.eSrijan.com/>.

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/usb.h>
```

```
#define MIN(a,b) (((a) <= (b)) ? (a) : (b))
#define BULK_EP_OUT 0x01
#define BULK_EP_IN 0x82
#define MAX_PKT_SIZE 512
```

```
static struct usb_device *device;
static struct usb_class_driver class;
static unsigned char bulk_buf[MAX_PKT_SIZE];
```

```
static int pen_open(struct inode *i, struct file *f)
{
    return 0;
}
static int pen_close(struct inode *i, struct file *f)
{
    return 0;
}
static ssize_t pen_read(struct file *f, char __user *buf, size_t
cnt, loff_t *off)
{
    int retval;
```